PROJECT VIVA VOCE & TECHNICAL DEMO GUIDE

CampusAI: Technical Architecture & Top 15 Viva Questions

Prepared for Final Project Evaluation & External Viva Voce Examination

Q1: What is the core problem that CampusAI solves?

Answer: Collegiate administrative offices are overwhelmed by repetitive inquiries regarding admissions, fees, and exams during working hours. CampusAI provides 24/7 instant, verified assistance via natural language, offloading administrative burdens and eliminating student queue delays.

Q2: What is your project's technology stack and why did you choose it?

Answer: We selected Python with Flask for the backend because it is lightweight, modular, and offers minimal computational overhead. SQLite was selected as an embedded, zero-configuration 3NF relational database. Semantic HTML5, CSS3 Variables, and Vanilla ES6 JavaScript were used for a responsive, high-performance client without heavy Node.js framework overhead.

Q3: How does your chatbot prevent artificial intelligence hallucinations?

Answer: CampusAI enforces a strict Dual-Tier response architecture. Tier 1 matches queries deterministically against indexed college FAQs in SQLite. If similarity is below 0.45, Tier 2 calls Gemini with a strict system prompt forbidding unverified facts. If the information is not in the knowledge base, the bot politely refers the student to official office contacts rather than fabricating answers.

Q4: Explain your NLP matching algorithm.

Answer: In services/faq_service.py, raw text is normalized (lowercased, punctuation removed, stop words filtered). Keywords are expanded through an educational synonym dictionary (e.g., 'fee' maps to tuition, cost, payment). It computes token overlap and Jaccard similarity against all stored FAQs. A composite score >= 0.45 returns the verified database answer.

Q5: What happens if the external AI API is down or there is no internet connection?

Answer: The system is designed with offline resilience. The Chatbot orchestrator wraps the AI service call in a try-except block. If the API fails, times out, or has no API key configured, it gracefully degrades to deterministic local FAQ matching without crashing or throwing HTTP 500 errors.

Q6: How is the database designed? Is it normalized?

Answer: Yes, the database is normalized to Third Normal Form (3NF). We have separate tables: admins (credentials), faqs (categorized Q&A;), chat_messages (chat logs with session UUIDs), and query_logs. All non-key attributes are fully functionally dependent on the primary key, with zero transitive dependencies.

Q7: How is the administrator authentication secured?

Answer: Passwords are never stored in plaintext. We utilize Werkzeug's PBKDF2:SHA256 password hashing with unique cryptographic salts. User sessions are secured using HTTP-only cookies and validated on every administrative request.

Q8: How are FAQ changes updated in real-time?

Answer: When an admin creates, edits, or deletes an FAQ via the dashboard, the changes are committed to SQLite via SQLAlchemy transactions. Because the chatbot queries the database dynamically on incoming messages, updates are reflected immediately without restarting the Flask server.

Q9: What automated testing did you perform?

Answer: We constructed a comprehensive test suite using Python's unittest framework with 23 test cases (TC-01 to TC-23) covering route status, keyword search, admin login, full CRUD lifecycles, and edge-case AI failure fallback. All 23 tests pass with 100% success rate.

Q10: What are the main limitations and future scope?

Answer: Current limitations include monolingual English NLP and manual FAQ entry. Future scope includes regional language support (Marathi, Hindi), voice/speech synthesis, webhook integration with WhatsApp/Telegram, and integration with the college ERP fee portal.